Artificial Intelligence
Introduction and examples

Stefaan Haspeslagh
Vives University of applied sciences – Association KU Leuven – Kortrijk, Belgium

Artificial Intelligence: introduction and examples

HEURISTIC SEARCH

Heuristic search

- Methods that use a heuristic function to provide specific knowledge about the problem
  - Heuristic functions
  - Hill climbing
  - Beam search
  - Greedy search

In blind search gingen we blindelings op zoek naar een oplossing in de zoekboom, we konden niet anders dan een conventie aannemen, paden genereren van links naar recht en dan invoegen in begin of op het einde, of zelfs invoegen op willekeurige plaatsen.
De tak die we kiezen in de boom kan dus een goede of een slechte zijn.
In dit hoofdstuk willen we beter proberen te doen. We willen kunnen inschatten welke tak, welk kind we best selecteren.
Om dit toe kunnen doen hebben we heuristische functies nodig.
Eenmaal we de heuristische functies hebben, dan kunnen we algoritmen bouwen zoals hill climbing, beam search en greedy search.

Heuristic functions

- To further improve the quality of the previous methods, we need to include problem-specific knowledge on the problem.
- How to do this in such a way that the algorithms remain generally applicable?
- Heuristic functions:
  - f: states → numbers
  - f(T): expresses the quality of the state T
    - Allows to express problem-specific knowledge
    - Can be imported in a generic way in the algorithms

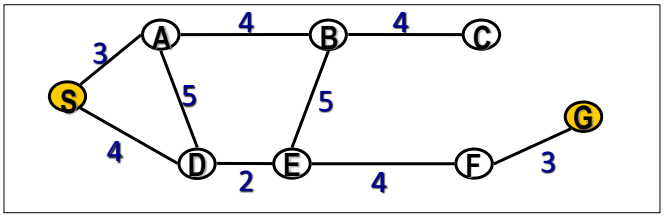
Met heuristische functies proberen we de kwaliteit van de vorige methode te verbeteren.
Om dit te kunnen doen moeten we kennis omtrent het probleem, dit noemt met domeinspecifieke kennis, kunnen gebruiken.

Omdat we niet telkens we een algoritme op een probleem willen toepassen, we het volledige algoritme zouden moeten herimplementeren, moeten we ervoor zorgen dat we de heuristische functies kunnen gebruiken vanuit een algoritme, maar dat het algoritme zelf overal toepasbaar blijft.

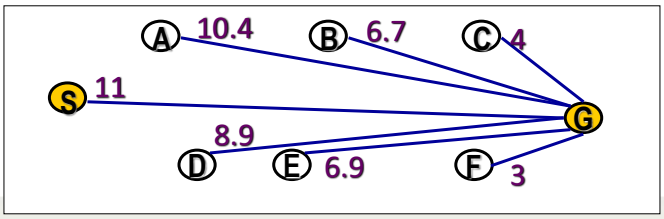
Een heuristische functie probeert 'een getal te plakken' op een toestand. Dat getal moet inschatten 'hoever' de toestand ligt van het doel dat moet bereikt worden. Dit is een zeer abstract begrip, de voorbeelden op de volgende slides moeten duidelijkheid brengen.
Vandaar de notatie f(T), de functie f probeert op elke toestand T een getal te plakken dat inschat hoe ver de toestand van het doel verwijderd is (of hoe goed de toestand dus is).

Heuristic functions: example 1

- Finding a route on a road map and that the NET below is the road map:



- Define f(T) = straight-line distance from T to G



In een eerste voorbeeld bekijken we een roadmap zoals we die in vorige topics ook al bekeken hebben. De getallen op de bogen tussen twee knopen zijn de afstanden tussen de knopen.

Een mogelijke heuristische functie is om vanuit elke node, de afstand in vogelvlucht tot het doel te nemen. Het uitdrukken van afstanden in vogelvlucht wordt is een gangbare dagdagelijkse praktijk wanneer mensen effectief afstanden tussen bv twee steden proberen uit te drukken op een kaart.
In plaats van een effectieve route te berekenen en alle afstanden op te tellen, gebruikt men die inschatting.

Let wel op: een afstand in vogelvlucht kan een zeer slechte inschatting zijn, kijk bv. naar knoop C. De afstand in vogelvlucht tussen C en G is bijvoorbeeld 4, maar de effectieve afstand (via de enige mogelijke route die kan genomen worden) is 16! De afstand in vogelvlucht is hier dus een enorme onderschatting!

Heuristic functions: example 2

Goal =

1	2	3
8		4
7	6	5

- f1(T) = the number correctly placed tiles on the board:

f1

1	3	2
8		4
5	6	7

= 4

- f2(T) = number or incorrectly placed tiles on board:
  - gives (rough!) estimate of how far we are from goal

f2

1	3	2
8		4
5	6	7

= 4

- Most often, 'distance to goal' heuristics are more useful!

Op deze slide proberen we heuristische functies op te stellen voor het 8-puzzle probleem (zie topic 3, state-space representation). We bekijken hier een lichte variant van de 8-puzzle, waar de oplossing hier rechtsboven getoond wordt.

Functie f1 telt het aantal juist geplaatste vakjes. In het voorbeeld zijn dat er 4.

Functie f2 telt het aantal verkeerd geplaatste vakjes, of hoe ver we verwijderd zijn van het doel.

Meestal zijn heuristiek die een afstand van het doel uitdrukken de meest bruikbare heuristieken.

Vraag: waarom telt f1 niet hoever we van het doel verwijderd zijn?

In het voorbeeld is dit wat verwarrend (niet per toeval), omdat beide functies de waarde 4 geven. Maar wat als er maar 3 vakjes juist geplaatst zijn?
Dan:
- f1(T) = 3
- f2(T) = 5 ! (Als er drie juist zijn op een totaal van 8, dan zijn er 5 verkeerd).

3 zou dan een onderschatting zijn omtrent hoever we van het doel verwijderd zijn, nl. 5!

Kortom, f2 is dus een inschatting hoe ver de toestand van de doeltoestand verwijderd is. Hoe groter f2, hoe verder van het doel. Hoe kleiner f2, hoe dichter bij de oplossing!

Heuristic functions: example 3

Goal =

1	2	3
8		4
7	6	5

- Manhatten distance

- f(3) = the sum of (the horizontal + vertical distance that each tile is away from its final destination):

f3

1	5	2
8		4
3	6	7

= 1 + 4 + 2 + 3 = 10

Een derde, veelgebruikte afstand is de manhattan afstand.
Deze afstand probeert te tellen hoever elk vakje van zijn doelpositie verwijderd is, dit is hoever in de horizontale + hoever in de verticale richting). Voor elk vakje telt met die afstand op en bekomt met dus de manhattan distance.

Bv. In het voorbeeld.
- Vakje 5 is 1 positie horizontaal en 2 positie verticaal verwijderd van zijn doel, dus afstand 3
- Vakje 2 is 1 positie horizontaal en 0 posities verticaal verwijderd van zijn doel, dus afstand 1
- Vakje 4 is 0 posities horizontaal en 0 posities verticaal verwijderd van zijn doel, dus afstand 0, hetzelfde voor vakje 8 en 6.
- Vakje 7 is 2 posities horizontaal en 0 posities verticaal verwijderd van zijn doel, dus afstand 2
- Vakje 3 is 2 posities horizontaal en 2 posities verticaal verwijderd van zijn doel, dus afstand 4

In totaal dus 3 + 1 + 2 + 4 = 10.

De manhattan distance in dit geval drukt het volgende uit, hoe groter de distance, hoe meer vakje ver verwijderd zijn van hun doelpositie. Hoe kleiner de manhattan distance, hoe meer vakjes op de juiste plaats staan of hoe dichter de meeste vakjes

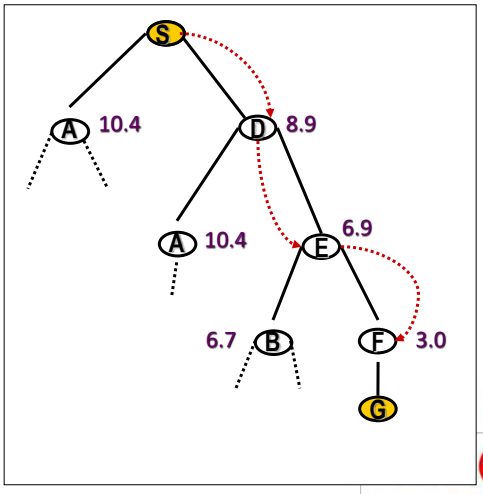
bij hun doelpositie staan.

Artificial Intelligence: introduction and examples

HEURISTIC SEARCH ALGORITHMS

Hill climbing

- A basic heuristic search method = depth-first + heuristic
- Example using straight line distance
- Perfortm depth-first, but:
  - Instead of left-to-right
  - FIRST select the child with the best heuristic value



Hill climbing is in essentie depth-first. De kinderen worden echter niet van links naar rechts gegenereerd en toegevoegd aan het begin van de QUEUE, maar voor we de kinderen toevoegen aan het begin van de QUEUE, sorteren we de nieuwe nodes eerst volgens waarde van de heuristische functie (eerst die met de kleinste waarde toevoegen, op het einde de nodes met hoogste waarde).

Op de figuur gebruiken we de heuristische functie en het netwerk van het eerste voorbeeld (met de afstand in vogelvlucht).

Hill climbing

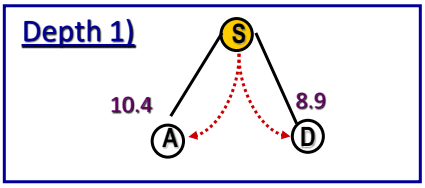
1. QUEUE ←path only containing the root;

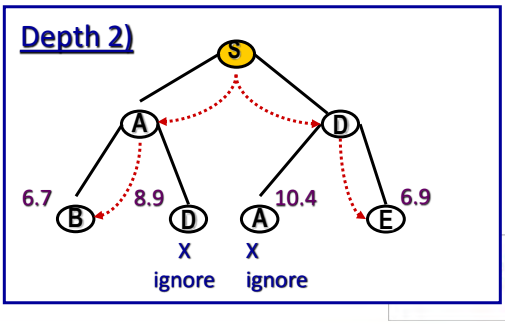
2. WHILE (QUEUE is not empty AND goal is not reached)
   - Remove the first path from the QUEUE
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Sort the new paths using the heuristic f
   - Add the new paths to the **front** of QUEUE

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

Beam search

- = breadth-first, but narrowing the width
- Assume a pre-fixed WIDTH (example: 2)
- Perform breadth-first, BUT:
  - Only keep the WIDTH best new nodes depending on heuristic at each new level.

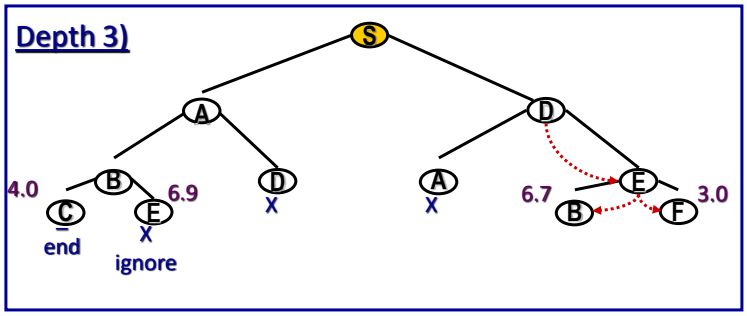


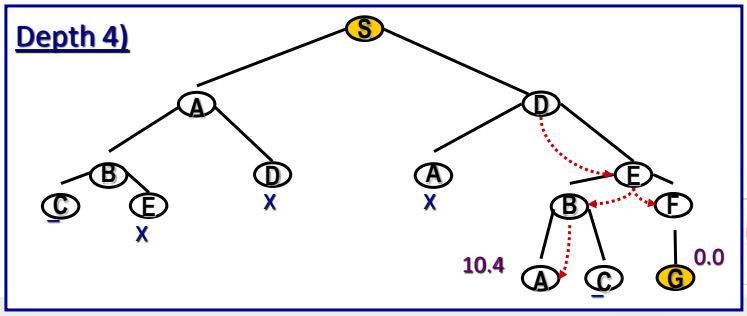


Bij beam search willen we heuristische functies inbouwen in het breedte-eerst algoritme.
We willen het aantal takken dat we bezoeken inperken. In plaats van alle kinderen van een knoop te beschouwen en dus toe te voegen aan de QUEUE, voegen we enkel een maximaal aantal knopen toe aan de QUEUE, dit is WIDTH.

Beam search

- Optimization:
  ignore leafs that are not goal nodes
- Algorithm:
  see exercises!





We kunnen dit algoritme nog een klein beetje verbeteren. Sommige kinderen zijn bladeren in de boom, en kunnen dus nooit tot een doel leiden. Daarom gaan we kinderen die bladeren zijn in de boom negeren.

Schrijf als uitstekende oefeninge voor thuis zelf de pseudocode van het algoritme op. In de oefeningen zal deze vraag ook voorkomen. Op termijn wordt een oplossing gegeven.

Hill climbing vs beam search

- Completeness
  - Hill climbing: yes
  - Beam search: no

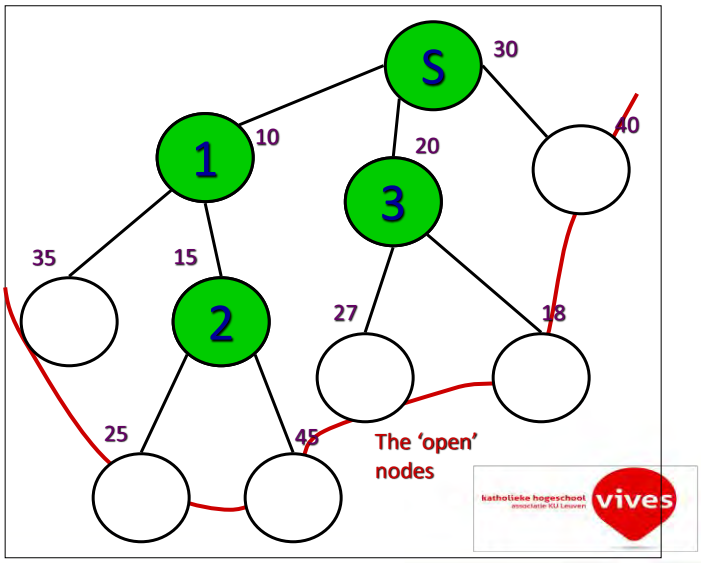
- Speed and memory:
  - Hill climbing: same as depth-first
  - Beam search:
    - Memory:
      - queue always has length WIDTH, memory usage is constant = WIDTH
    - Time:
      - O(WIDTH * b * m) if goal at depth m
      - O(WIDTH * b * d) if no solution found

Probeer deze analyse zelf te bekomen! Als je dit lukt, dan snap je zeker de analysestukken rond diepte-eerst en breedte-eerst.

Stel vragen indien je niet tot deze uitkomsten komt! Je moet alle uitkomsten kunnen uitleggen en verklaren!

Greedy search

- Always expand the heuristically best nodes first.
- At each step, select the node with the best (in this case: lowest) heuristic value.



Hill-climbing blijft de boom tak per tak bestuderen, afhankelijk van de f-waarde.
Beam-seach blijft de boom niveau per niveau bestuderen, afhankelijk van de f-waarde.

Greedy-search sluit beter aan bij non-deterministic search, waar we een nieuwe pad gelijk waar in de QUEUE invoegen. We kunnen dus bij non-deterministic search niet meer zeker zijn of de QUEUE tak per tak of niveau per niveau bestudeerd wordt.
Bij greedy search zullen we de nieuw gegenereerde paden gewoon toevoegen aan de QUEUE en dan de volledige QUEUE hersorteren volgens f-waarde. Hier kunnen we dus ook niet meer zeker zijn of de QUEUE tak per tak of niveau per niveau bestudeerd wordt.

Greedy search

1. QUEUE ←path only containing the root;

2. WHILE (QUEUE is not empty AND goal is not reached)
   - Remove the first path from the QUEUE
   - Create new paths (to all children);
   - Reject the new paths with loops;
   - Add the new paths and sort the entire QUEUE using heuristic f

3. IF (goal reached)
   - THEN SUCCESS
   - ELSE failure

Questions

?

Coordinates:

Dr. Stefaan Haspeslagh

A.I. Research & Education

Vives University of applied sciences – KU Leuven Association

E-mail: stefaan.haspeslagh@vives.be